```
(*
                            CS51 Lab 19
                    2-dimensional cellular automata
 *)
(*
                                SOLUTION
 *)

module G = Graphics ;;

    (*************************************************************
    Do not change either of the two module type signatures in this
    file. Doing so will likely cause your code not to compile
    against our unit tests.
    *************************************************************)

(*.....................................................................
  Specifying automata

  A cellular automaton (CA) is a square grid of cells each in a
  specific state. The grid is destructively updated according to an
  update rule. The evolving state of the CA can be visualized by
  displaying the grid in a graphics window.

  In this implementation, a particular kind of automaton, with its
  state space and update function, is specified by a module satisfying
  the `AUT_SPEC` signature. *)

module type AUT_SPEC =
  sig
    (* Automaton parameters *)
    type state                  (* the space of possible cell states *)
    val initial : state         (* the initial (default) cell state *)
    val grid_size : int         (* width and height of grid in cells *)
    val update : (state array array) -> int -> int -> state
                                (* update grid i j -- returns the new
                                   state for the cell at position `i, j`
                                   in `grid` *)
    val initialize : state array array -> unit
                                (* initialize grid -- Fills `grid` with the
                                   automaton's chosen starting configuration *)
    (* Rendering parameters *)
    val name : string           (* a display name for the automaton *)
    val side_size : int         (* width and height of cells in pixels *)
    val cell_color : state -> G.color
                                (* color for each state *)
    val legend_color : G.color (* color to render legend *)
    val font : string option    (* optional font for legend as per
                                   Graphics module *)
    val render_frequency : int (* how frequently grid is rendered
                                   (in ticks) *)
  end ;;
```

```
(*.....................................................................
  Automata functionality

  Implementations of cellular automata provide for the following
  functionality:

  * A current (mutable) grid that can be modified (`replace_grid`).

  * Creating additional fresh grids (`fresh_grid`).

  * Initializing the graphics window in preparation for rendering
    (`graphics_init`).

  * Mapping the automaton's update function simultaneously over all
    cells of the current grid (`update_grid`).

  * Running the automaton using a particular update function, and
    rendering it as it evolves (`run_grid`).

  as codified in the `AUTOMATON` signature.

  Note the difference between the argument structure of functions in
  `life.ml`, where the grid is passed in as argument to several
  functions, as compared to the approach here, where there is a single
  mutable current grid that is updated by the functions, so that no
  grid argument is required.
 *)

module type AUTOMATON =
  sig
    (* state -- Possible states that a grid cell can be in *)
    type state

    (* grid -- 2D grid of cell states *)
    type grid = state array array

    (* current_grid -- The current grid of the appropriate size as per
       the spec, which is initialized with all cells being in the
       initial state. It can be modified directly or by `replace_grid`
       or updated with the automaton's update rule. *)
    val current_grid : grid

    (* fresh_grid () -- Returns a fresh grid of the appropriate size
       as per the spec, initialized with all cells being in the
       initial state. *)
    val fresh_grid : unit -> grid

    (* initialized_grid () -- Returns a fresh grid of the appropriate
       size as per the spec, initialized by calling the spec's
       `initialize` function. *)
    val initialized_grid : unit -> grid

    (* replace_grid new_grid -- Destructively changes the current grid
```

```
107         (`current_grid`, above) to `new_grid`. *)
108       val replace_grid : grid -> unit
109
110       (* graphics_init () -- Initialize the graphics window to the
111          correct size and other parameters. Auto-synchronizing is off,
112          so our code is responsible for flushing to the screen upon
113          rendering. *)
114       val graphics_init : unit -> unit
115
116       (* update_grid () -- Updates the current grid one "tick" by
117          updating each cell simultaneously as per the CA's update
118          rule. *)
119       val update_grid : unit -> unit
120
121       (* run_grid grid -- Initializes the current grid with `grid` and
122          repeatedly updates it using the automaton's update rule,
123          rendering the grid state to the graphics window until a key is
124          pressed. Assumes graphics has been initialized with
125          `graphics_init`. *)
126       val run_grid : grid -> unit
127     end ;;
128
129   (*.....................................................................
130     Implementing automata based on a specification
131
132     Given an automaton specification (satisfying `AUT_SPEC`), the
133     `Automaton` functor delivers a module satisfying `AUTOMATON` that
134     implements the specified automaton. *)
135
136   module Automaton (Spec : AUT_SPEC)
137          : (AUTOMATON with type state = Spec.state) =
138     struct
139       type state = Spec.state
140       type grid = Spec.state array array
141
142       (* fresh_grid () -- See module type documentation *)
143       let fresh_grid (() : unit) : grid =
144         Array.make_matrix Spec.grid_size Spec.grid_size Spec.initial
145
146       (* current -- The current grid, which is destructively updated
147          after each tick; initialized with the initial state. *)
148       let current_grid =
149         fresh_grid ()
150
151       let initialized_grid () : grid =
152         let mat = fresh_grid () in
153         Spec.initialize mat;
154         mat ;;
155
156       (* replace_grid -- See module type documentation *)
157       let replace_grid (new_grid : grid) : unit =
158         for i = 0 to Spec.grid_size - 1 do
159           for j = 0 to Spec.grid_size - 1 do
160             current_grid.(i).(j) <- new_grid.(i).(j)
```

```
161         done
162       done
163
164     (* graphics_init -- See module type documentation *)
165     let graphics_init () : unit =
166       G.open_graph "";
167       G.resize_window (Spec.grid_size * Spec.side_size)
168                       (Spec.grid_size * Spec.side_size);
169       (match Spec.font with
170        | None -> ()
171        | Some fontspec -> G.set_font fontspec);
172       G.auto_synchronize false
173
174     (* update_grid -- See module type definition. *)
175     let update_grid =
176       (* use a single static temp grid across invocations *)
177       let temp_grid = fresh_grid () in
178       fun () -> for i = 0 to Spec.grid_size - 1 do
179                   for j = 0 to Spec.grid_size - 1 do
180                     temp_grid.(i).(j) <- Spec.update current_grid i j
181                   done
182                 done;
183                 replace_grid temp_grid
184
185     (* render_grid grid legend -- Renders the `grid` to the already
186        initialized graphics window including the textual `legend` *)
187     let render_grid (grid : grid) (legend : string) : unit =
188       (* draw the grid of cells *)
189       for i = 0 to Spec.grid_size - 1 do
190         for j = 0 to Spec.grid_size - 1 do
191           G.set_color (Spec.cell_color grid.(i).(j));
192           G.fill_rect (i * Spec.side_size) (j * Spec.side_size)
193                       (Spec.side_size - 1) (Spec.side_size - 1)
194         done
195       done;
196       (* draw the legend *)
197       G.moveto (Spec.side_size * 2) (Spec.side_size * 2);
198       G.set_color Spec.legend_color;
199       G.draw_string legend;
200       (* flush to the screen *)
201       G.synchronize ()
202
203     (* run_grid -- See module type documentation *)
204     let run_grid (initial_grid : grid) : unit =
205       replace_grid initial_grid;
206       let tick = ref 0 in
207       render_grid current_grid (Printf.sprintf "%s: tick %d" Spec.name !tick);
208       ignore (G.read_key ());    (* pause at start until key is pressed *)
209       while not (G.key_pressed ()) do
210         update_grid ();
211         tick := succ !tick;
212         if !tick mod Spec.render_frequency = 0 then
213           render_grid current_grid (Printf.sprintf "%s: tick %d" Spec.name !tick);
```

```
214         done;
215         ignore (G.read_key ());    (* clear the keypress *)
216         ignore (G.read_key ());    (* await another keypress *)
217         G.close_graph () ;;
218     end ;;
```